

Estruturas de Dados

Análise de Algoritmos

Universidade Federal do Rio Grande do Norte

Introdução

- Os algoritmos são a parte mais importante e durável da computação, pois podem ser estudados de maneira **independente da linguagem e da máquina**.
- Isso significa que precisamos de técnicas que nos permitam **comparar a eficiência** dos algoritmos independente do contexto de uso.
- Analisar um algoritmo significa tentar prever os **recursos** que esse algoritmo necessitará quando for executado.
 - Tempo de execução
 - Espaço na memória

Introdução

- Para aferir a eficiência podemos adotar duas abordagens:
- **Análise empírica** – executando o programa com entradas de tamanho e informações variadas.
 - os resultados podem não ser conclusivos
 - comparação de algoritmos obriga a usar **igual software e hardware**
- **Análise teórica** – estabelecendo um modelo de computação
 - usa uma descrição de mais alto nível do algoritmo em vez da implementação.
 - depende de um **computador hipotético**.
 - permite avaliar eficiência de forma independente do ambiente de hardware/software

Regras do Computador Hipotético

- Cada operação simples (+, *, -, =, if, call) leva exatamente um **intervalo de tempo**.
- Iterações e sub-rotinas não são considerados operações simples. Em vez disso, eles são a **composição de muitas operações de uma única etapa**.
- Cada acesso à memória leva exatamente **um intervalo de tempo**.

Uso do Computador Hipotético

- No computador hipotético medimos o **tempo de execução contando o número de etapas** que um algoritmo executa em uma determinada instância do problema.

```
function maior(array){  
    let maior = array[0];  
    let i = 1;  
    while(i < array.length){  
        if(maior < array[i]){  
            maior = array[i];  
        }  
        i = i+1;  
    }  
    return maior;  
}
```

Uso do Computador Hipotético

- No computador hipotético medimos o **tempo de execução contando o número de etapas** que um algoritmo executa em uma determinada instância do problema.

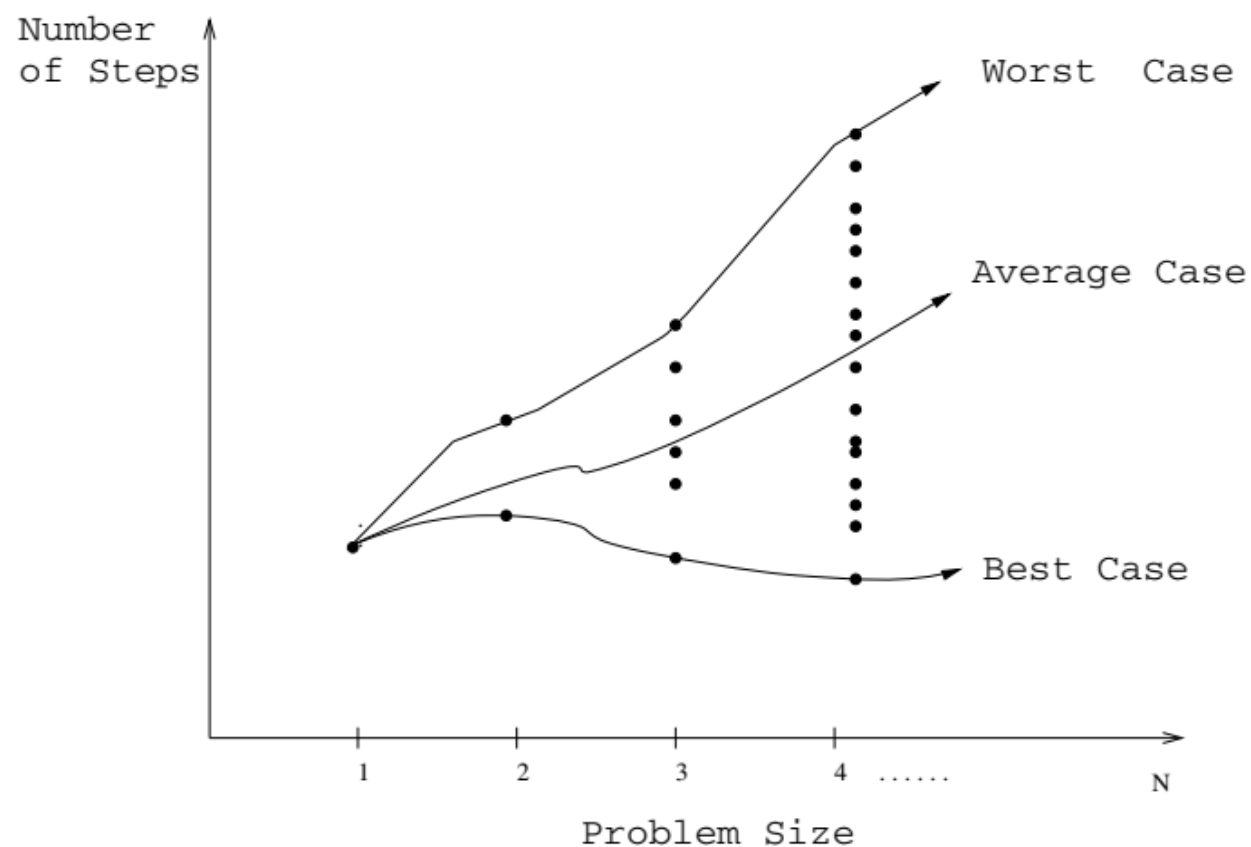
```
function maior(array){  
    let maior = array[0]; _____ 2 operações  
    let i = 1; _____ 1 operação  
    while(i < array.length){ _____ n operações  
        if(maior < array[i]){ _____ 2 operações  
            maior = array[i]; _____ 2 operações  
        }  
        i = i+1; _____ 2 operações  
    }  
    return maior; _____ 1 operação  
}
```

n-1 vezes

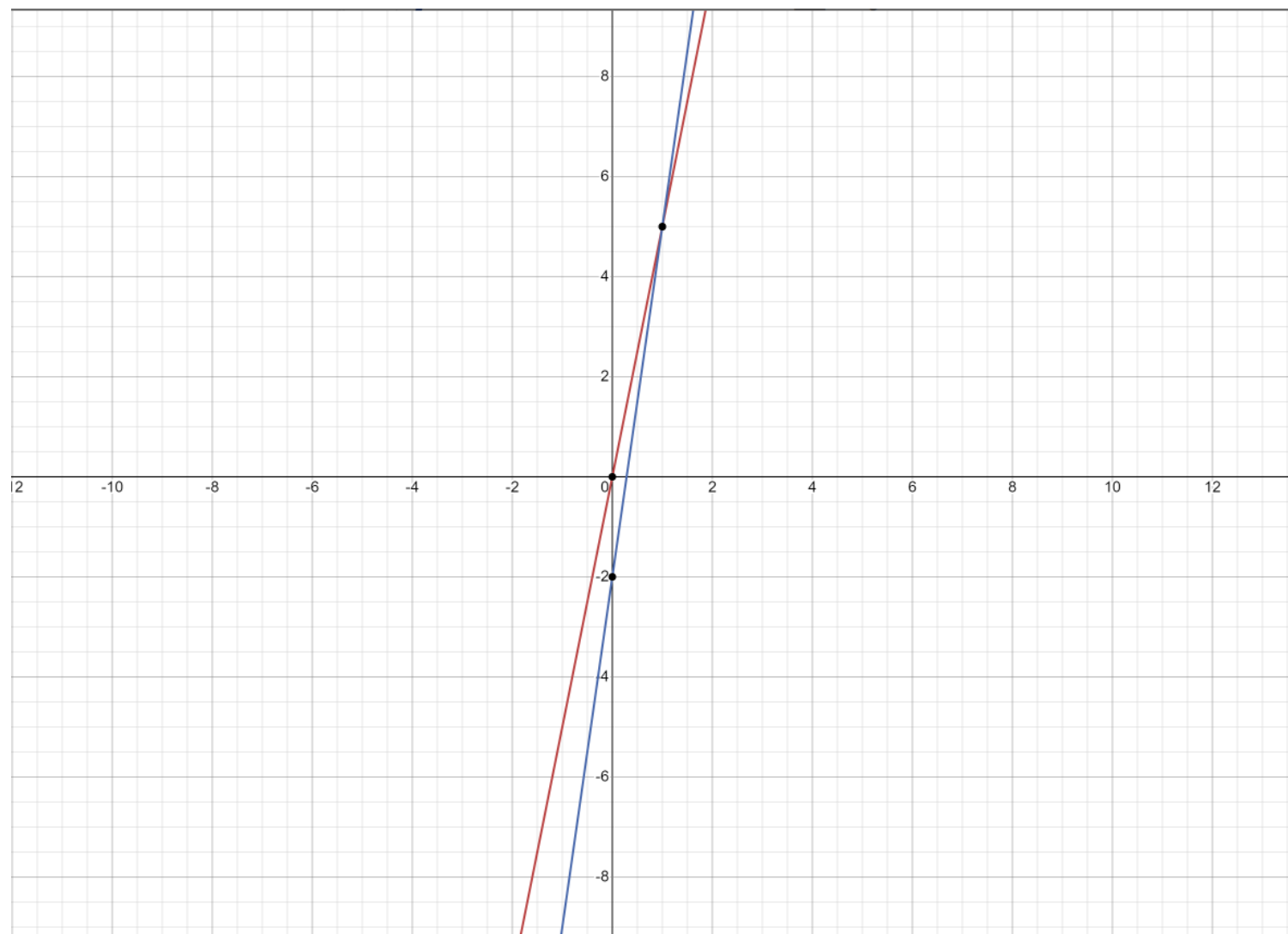
Casos possíveis

- **Melhor caso** (array[0] é o maior elemento)
 - $T(n) = 2 + 1 + n + 4(n - 1) + 1 = 5n$ operações
- **Pior caso** (array[array.length) é o maior elemento)
 - $T(n) = 2 + 1 + n + 6(n - 1) + 1 = 7n - 2$ operações
- **Caso médio**
 - Depende da distribuição da entrada.
 - É necessário usar a teoria de probabilidades.

Casos possíveis



Casos possíveis



$$T(n) = 5n$$

$$T(n) = 7n - 2$$

Casos possíveis

Pior caso

A complexidade de pior caso do algoritmo é a **função definida pelo número máximo de passos dados** em qualquer instância de tamanho n . Isto representa a curva que passa pelo ponto mais alto de cada coluna.

Melhor caso

A complexidade do melhor caso do algoritmo é a **função definida pelo número mínimo de passos** dados em qualquer instância de tamanho n . Isso representa o curva passando pelo ponto mais baixo de cada coluna.

Caso Médio

A complexidade de caso médio do algoritmo, que é a função definida pelo **número médio de etapas** em todas as instâncias de tamanho n .

Casos possíveis

- A análise do pior caso acaba sendo mais útil que as outras análises.
- A depender da função do pior caso, podemos, por exemplo, determinar o tamanho máximo de uma entrada.
- O tempo de execução, $T(n)$, pode ser afetado pela alteração do ambiente hardware/software, mas o mesmo não acontece se considerarmos a **taxa de crescimento de $T(n)$** .

Taxa de Crescimento

- O exemplo maior() mostrava $T(n)$ limitado por 2 funções lineares em n , significando que o tempo de execução varia na mesma proporção que n .
 - $T(n) = 5n$ (melhor caso)
 - $T(n) = 7n - 2$ (pior caso)
- Diz-se que o **crescimento de $T(n)$ é linear**.

Taxa de Crescimento

- Considerando que $T(n)$, hipoteticamente, é 1 nanosegundo a tabela abaixo mostra como diferentes funções lidariam com diferentes tamanhos de entrada.

n	$f(n)$	$\lg n$	n	$n \lg n$	n^2	2^n	$n!$
10		0.003 μs	0.01 μs	0.033 μs	0.1 μs	1 μs	3.63 ms
20		0.004 μs	0.02 μs	0.086 μs	0.4 μs	1 ms	77.1 years
30		0.005 μs	0.03 μs	0.147 μs	0.9 μs	1 sec	8.4×10^{15} yrs
40		0.005 μs	0.04 μs	0.213 μs	1.6 μs	18.3 min	
50		0.006 μs	0.05 μs	0.282 μs	2.5 μs	13 days	
100		0.007 μs	0.1 μs	0.644 μs	10 μs	4×10^{13} yrs	
1,000		0.010 μs	1.00 μs	9.966 μs	1 ms		
10,000		0.013 μs	10 μs	130 μs	100 ms		
100,000		0.017 μs	0.10 ms	1.67 ms	10 sec		
1,000,000		0.020 μs	1 ms	19.93 ms	16.7 min		
10,000,000		0.023 μs	0.01 sec	0.23 sec	1.16 days		
100,000,000		0.027 μs	0.10 sec	2.66 sec	115.7 days		
1,000,000,000		0.030 μs	1 sec	29.90 sec	31.7 years		

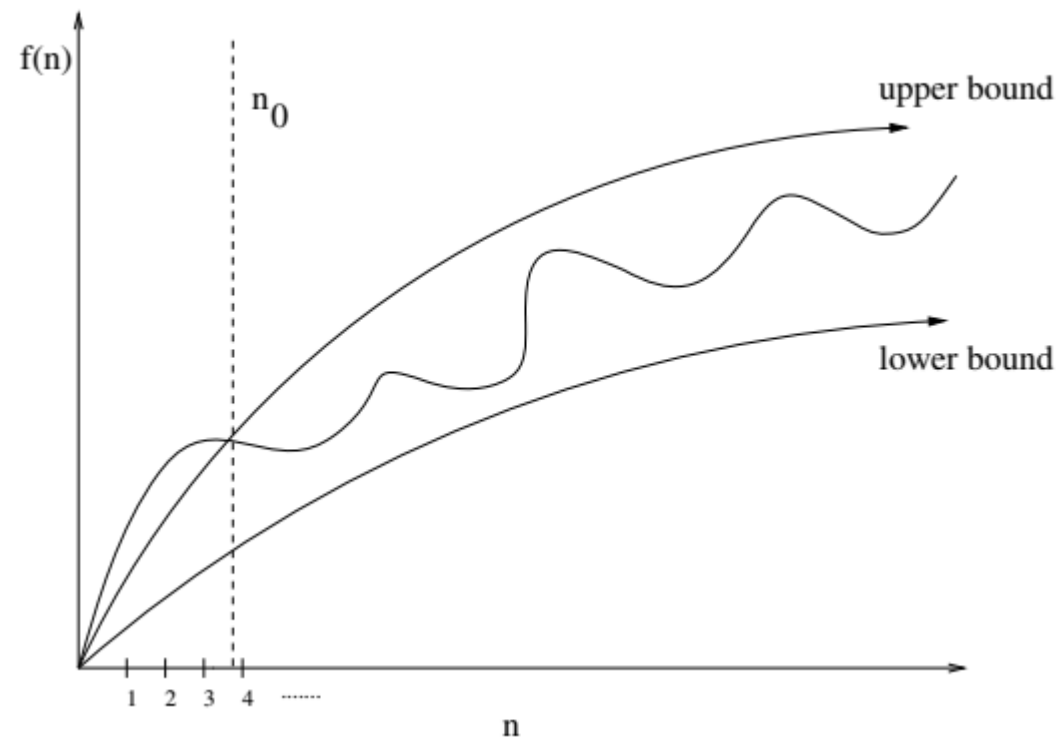
Taxa de Crescimento

- Para valores pequenos de n , praticamente qualquer algoritmo custa pouco para ser executado.
- Neste caso, a escolha do algoritmo tem pouquíssima influência em problemas de tamanho pequeno.

Comportamento Assintótico

- A análise de algoritmos deve ser realizada para valores **grandes de n** .
- Necessário estudar o **comportamento assintótico** das funções de custo.
- O comportamento assintótico de $f(n)$ representa **o limite do comportamento do custo quando n cresce**.

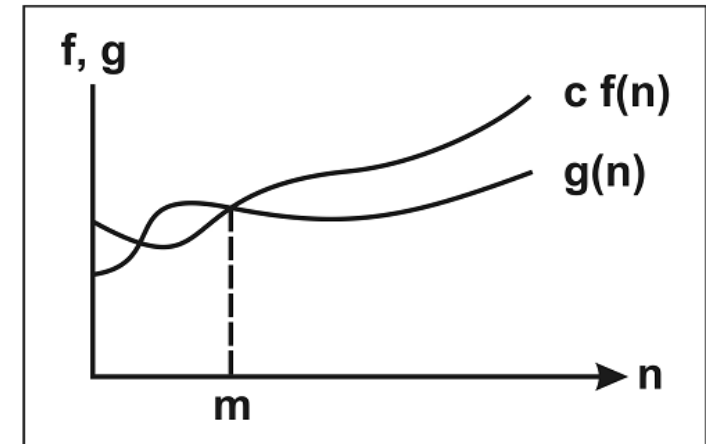
Comportamento Assintótico



Comportamento Assintótico

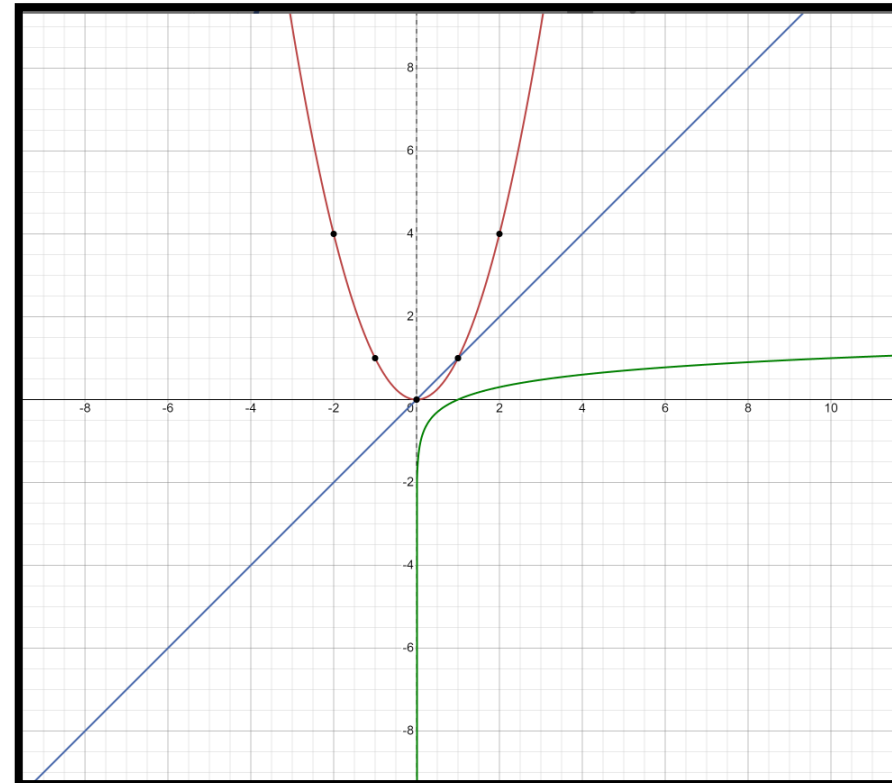
- Uma função $f(n)$ domina assintoticamente outra função $g(n)$ se:
- Existem duas constantes positivas **c** e **m** tais que
para $n \geq m$,

$$|g(n)| \leq c |f(n)|$$



Comportamento Assintótico

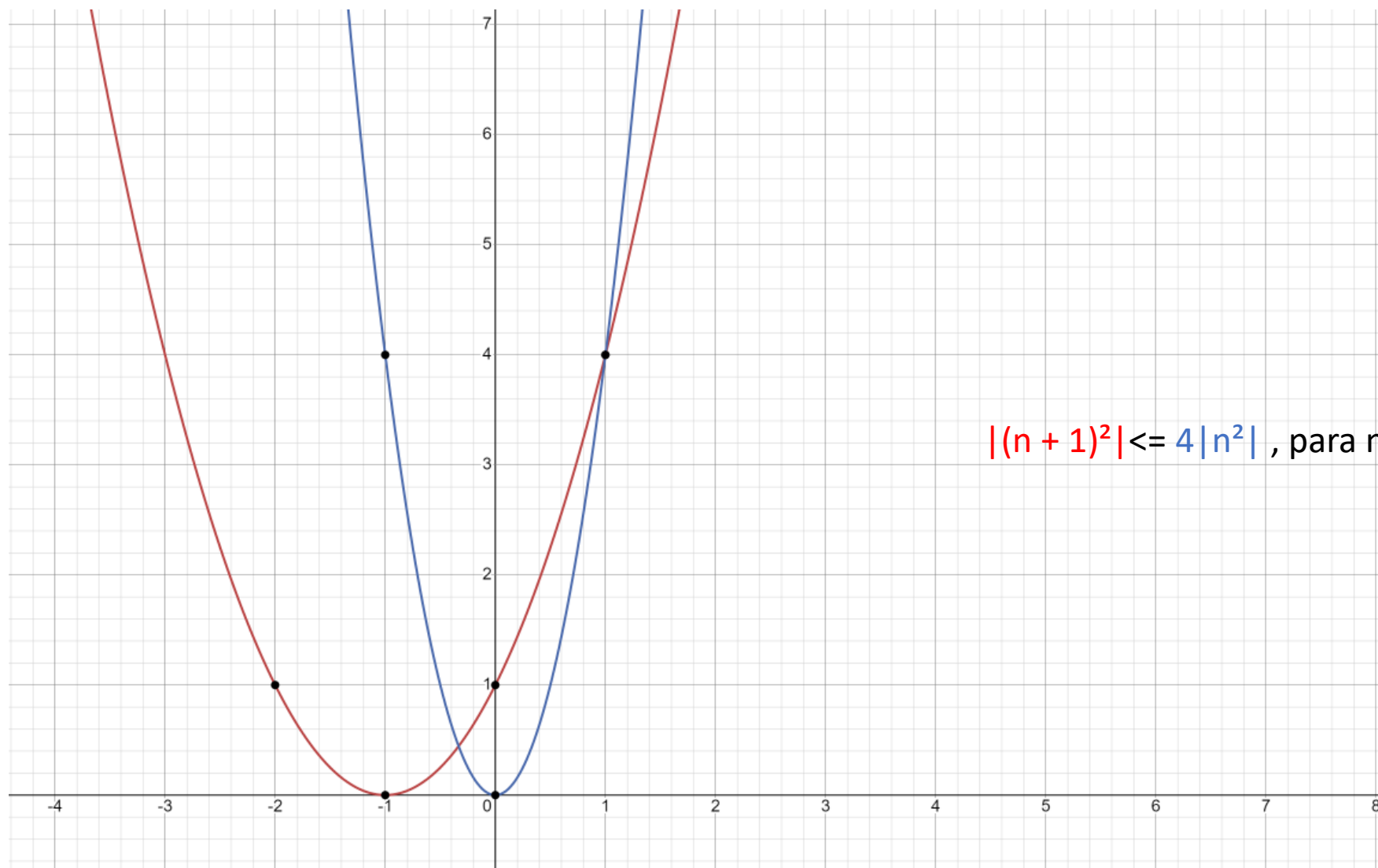
- Seja $T(n)$,
- Observe a demonstração para os limites:
 - $f(n) = n^2$
 - $f(n) = n$
 - $f(n) = \log n$



Comportamento Assintótico

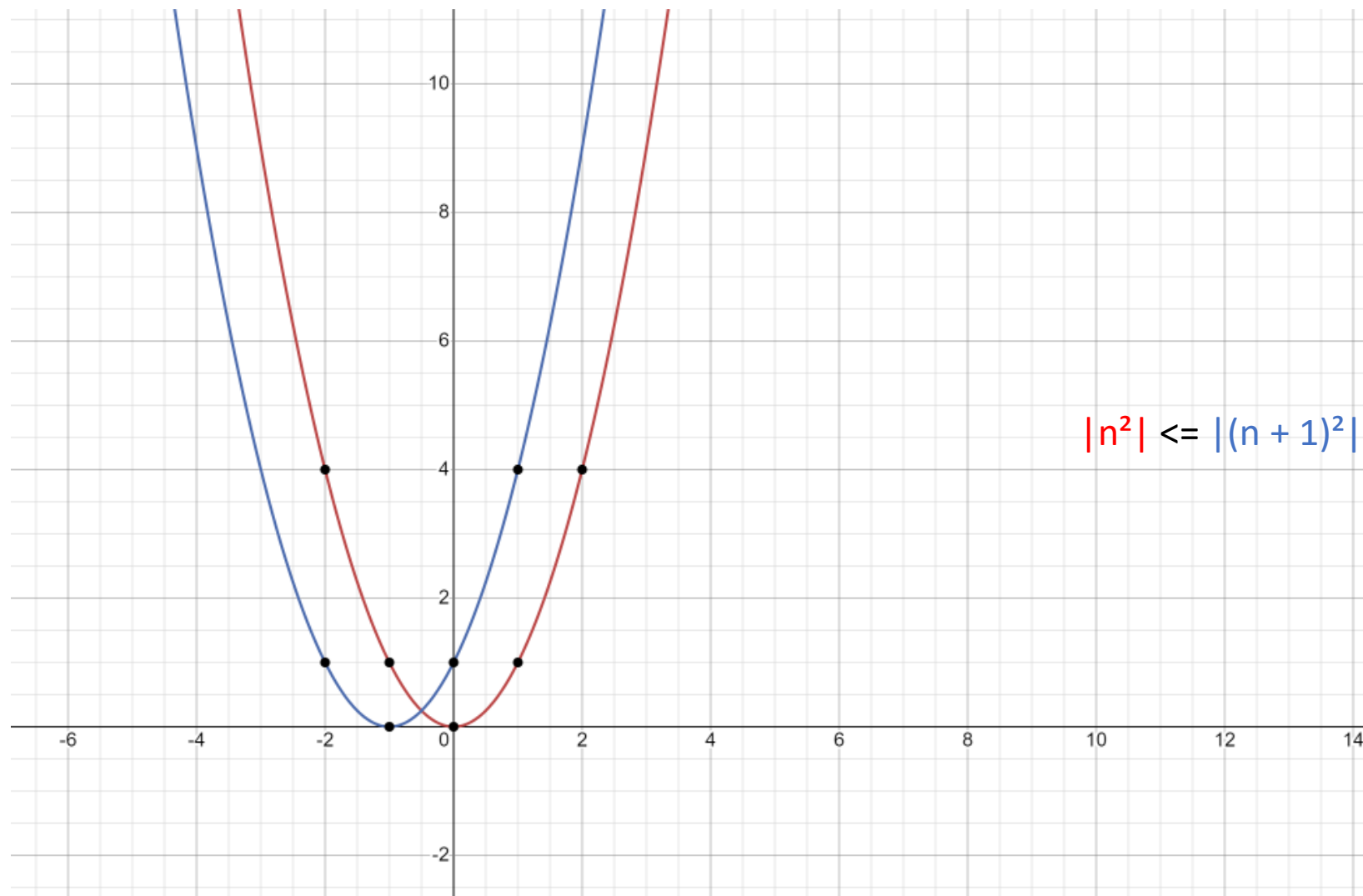
- Exemplo,
- Sejam $g(n) = (n + 1)^2$ e $f(n) = n^2$
- As funções $g(n)$ e $f(n)$ dominam assintoticamente uma à outra, desde que:
 - $|(n + 1)^2| \leq 4|n^2|$, para $n \geq 1$
 - $|n^2| \leq |(n + 1)^2|$, para $n \geq 0$

$$|(n + 1)^2| \leq 4|n^2|, \text{ para } n \geq 1$$



$$|(n + 1)^2| \leq 4|n^2|, \text{ para } n \geq 1$$

$$|n^2| \leq |(n + 1)^2|, \text{ para } n \geq 0$$



$$|n^2| \leq |(n + 1)^2|, \text{ para } n \geq 0$$

Notação O (big oh)

- As complexidades de **tempo de melhor, pior e caso médio para qualquer algoritmo** são funções numéricas sobre o tamanho das possíveis instâncias do problema.
- É muito difícil trabalhar **precisamente** com essas funções, porque elas tendem a requerer muitos detalhes para especificar com precisão.

Notação O (big oh)

$$T(n) = 12754n^2 + 4353n + 834 \lg^2 n + 13546$$

- Uma função como essa seria claramente difícil de trabalhar e de se determinar.
- Além disso, ela nos oferece poucas informações extras já que apenas observando-a sabemos poderíamos dizer que *"o tempo cresce quadraticamente com n."*

Notação O (big oh)

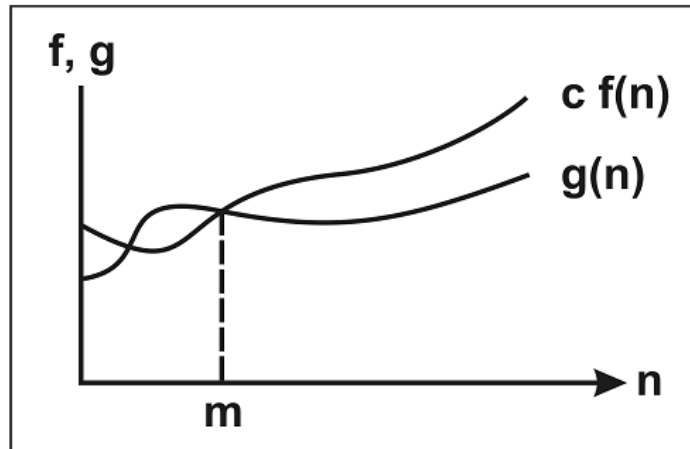
- É muito mais fácil falar em termos de **limites superiores e inferiores** de funções de complexidade de tempo usando a notação **Big Oh**.
- Big Oh simplifica nossa análise, ignorando níveis de detalhe que não afetam nossa comparação de algoritmos.
- A notação Big Oh ignora a diferença entre constantes multiplicativas. As funções $f(n) = 2n$ e $g(n) = n$ são idênticas na análise de Big Oh.

Notação O (big oh)

- Escrevemos $g(n) = O(f(n))$ para expressar que $f(n)$ domina assintoticamente $g(n)$.
- Lê-se $g(n)$ é da ordem de no máximo $f(n)$.
- Exemplo:
- Quando dizemos que o tempo de execução $T(n)$ de um programa é $O(n^2)$, significa que existem constantes c e m tais que, para valores de $n \geq m$, $T(n) \leq cn^2$

Notação O (big oh)

- Exemplo gráfico de dominação assintótica que ilustra a notação O.
- $f(n)$ domina assintoticamente a função $g(n)$.



Propriedades

- $f(n) = O(f(n))$
- $c * O(f(n)) = O(f(n))$, $c = \text{constante}$
- $O(f(n)) + O(f(n)) = O(f(n))$
- $O(O(f(n))) = O(f(n))$
- $O(f(n)) + O(g(n)) = O(\max(f(n), g(n)))$
- $O(f(n)) * O(g(n)) = O(f(n) * g(n))$
- $f(n) * O(g(n)) = O(f(n) * g(n))$

Referências

- CORMEN, Thomas H et al. **Algoritmos: teoria e prática**. Rio de Janeiro: Elsevier, 2012. 926 p. ISBN: 9788535236996.
- ASCENCIO, Ana Fernanda Gomes. **Estruturas de dados: algoritmos, análise da complexidade e implementações em Java e C/C++**. São Paulo: Pearson, c2010. 432 p. ISBN: 9788576052216, 978857605816.
- PIVA JÚNIOR, Dilermando (et al). **Estrutura de dados e técnicas de programação**. 1. ed. Rio de Janeiro, RJ: Campus, 2014. 399 p. ISBN: 9788535274370.
- SKIENA, S. S. **The Algorithm Design Manual**. Second Edition. Springer London. 2008.
- Reinaldo Faria. Notas de aula. Disponível em:
[http://www.decom.ufop.br/reinaldo/site_media/uploads/2013-02-bcc202/aula_04_-_analise_de_algoritmos_\(parte_1\)_v2.pdf](http://www.decom.ufop.br/reinaldo/site_media/uploads/2013-02-bcc202/aula_04_-_analise_de_algoritmos_(parte_1)_v2.pdf)

Referências

- FERRARI, Roberto et al. **Estruturas de dados com jogos**. 1. ed. Rio de Janeiro: Elsevier, 2014. 259p. ISBN: 9788535278040.
- GRONER, Loiane. **Estruturas de dados e algoritmos em Javascript**: aperfeiçoe suas habilidades conhecendo estruturas de dados e algoritmos clássicos em JavaScript. São Paulo: Novatec, 2017. 302 p. ISBN: 9788575225530.
- SZWARCFITER, Jayme Luiz; MARKENZON, Lilian. **Estruturas de dados e seus algoritmos**. 3. ed. Rio de Janeiro: LTC, 2010. xv, 302 p. ISBN: 9788521617501.
- GOODRICH, Michael T; TAMASSIA, Roberto. **Estruturas de dados e algoritmos em Java**. 5. ed. Porto Alegre: Bookman, 2013. xxii, 713 p. ISBN: 9788582600184.
- GUIMARÃES, Ângelo M. **Algoritmos e estruturas de dados**. LTC, 1994.